

USD-Goal Crowdfunding with Automatic Refunds

Scaffold-HBAR External Template · Hedera Testnet · Oracle-backed · HBAR-only contributions

Version: 1.0 **Date:** 4 October 2026 **Status:** Locked MVP **Network:** Hedera Testnet (chainId 296)

Audience: coding agent / developer implementing the template end-to-end. This document is the single source of truth. Do not invent features outside this scope. Prefer exact names, addresses, and semantics below.

1. Summary

Build a Scaffold-HBAR external template for a **USD-denominated crowdfunding campaign** on Hedera. An organizer sets a goal in USD and a deadline. Contributors send **only testnet HBAR**. After the deadline, anyone can call **finalize()**, which reads the Chainlink HBAR/USD feed **once** and decides whether the goal was met. If met, the organizer withdraws the pot. If not, each contributor claims a full refund of their HBAR.

Real use cases: community drives, school projects, emergency funds, small creative campaigns.

Scaffold command: `npm create scaffold-hbar@latest --template <org/repo>`

Closest collision: price-triggered escrow templates. This product is *pooled contributions with per-contributor refunds* — different flow. State that clearly in README.

2. Goals and Non-Goals

2.1 Goals

- One deployed contract = one campaign (constructor sets goalUsd, deadline, feed, maxAge).
- Contributors need only testnet HBAR — no second token, no association, no WHBAR.
- Oracle is load-bearing: without a valid HBAR/USD read, goalMet has no meaning.
- Simple one-page UI: progress bar, time left, Contribute / Finalize / Withdraw / Refund.
- Pass eligibility gate: template.json, README, AGENTS.md, clean install/build, MIT, one HashScan proof.
- Reentrancy-safe withdraw and refund; unit tests for wei/USD math boundaries.

2.2 Non-Goals (do not implement in MVP)

- Multiple campaigns in one contract — use constructor; deploy script may deploy two instances for demo.
- Snapshotting USD on every contribute (stale-feed risk + wrong semantics if HBAR moves).
- Partial withdrawals, early cancel, or organizer cancel mid-campaign.
- HCS receipts and supporter badge NFTs (stretch only if core done with spare time).
- Mainnet hardening, audits, production fees.
- Any DEX, SaucerSwap, LP, or multi-token flow.

3. Locked MVP Contract Interface

Implement exactly this surface. Do not add extra public mutators unless required for a documented UI read.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface AggregatorV3Interface {
    function latestRoundData() external view returns (
        uint80 roundId, int256 answer, uint256 startedAt,
        uint256 updatedAt, uint80 answeredInRound
    );
    function decimals() external view returns (uint8);
}

/// @title UsdGoalCrowdfund — one campaign per deployment
/// @notice Goal in USD (8 decimals). Contributions are native HBAR only.
contract UsdGoalCrowdfund {
    address public immutable organizer;
    uint256 public immutable goalUsd; // 8 decimals ($100 = 100e8)
    uint256 public immutable deadline; // unix seconds
    AggregatorV3Interface public immutable feed;
    uint256 public immutable maxAge; // seconds

    mapping(address => uint256) public contributions; // 18-decimal HBAR
```

```

uint256 public totalRaised;
bool public finalized;
bool public goalMet;
bool public withdrawn;

event Contributed(address indexed contributor, uint256 amount, uint256 totalRaised);
event Finalized(bool goalMet, int256 priceAnswer, uint256 updatedAt, uint256 totalRaised);
event Withdrawn(address indexed organizer, uint256 amount);
event Refunded(address indexed contributor, uint256 amount);

constructor(uint256 goalUsd_, uint256 deadline_, address feed_, uint256 maxAge_);

function contribute() external payable;
function finalize() external;
function withdraw() external;
function refund() external;

function timeLeft() external view returns (uint256);
function isOpen() external view returns (bool);
function previewUsdValue(uint256 hbarAmount) external view returns (uint256 usd8);
}

```

3.1 Function semantics (must follow)

Function	Rules
contribute()	Require !finalized, block.timestamp < deadline, msg.value > 0. Add to contributions[msg.sender] and totalRaised. Emit Contributed.
finalize()	Require !finalized and block.timestamp >= deadline. Read latestRoundData(). Require answer > 0, updatedAt freshness <= maxAge, answeredInRound >= roundId. Compute usdRaised from totalRaised and answer (see Units). Set goalMet = (usdRaised >= goalUsd), finalized = true. Emit Finalized.
withdraw()	Require msg.sender == organizer, finalized, goalMet, !withdrawn. Set withdrawn = true, then transfer totalRaised to organizer. Emit Withdrawn.
refund()	Require finalized, !goalMet, contributions[msg.sender] > 0. amount = contributions[msg.sender]; contributions[msg.sender] = 0; then transfer amount. Emit Refunded.

3.2 Why price is read once in finalize() — do not change

- Testnet Chainlink can sit stale for hours; per-contribute snapshots break when one read is stale.
- If HBAR drops after contributions, per-contribute USD snapshots can claim goal met while the pot is worth less.
- Single read at finalize is simpler to document. State volatility clearly in README.

4. Oracle (validated 4 Oct 2026)

Field	Value
Feed address (Hedera testnet)	0x59bC155EB6c6C415fE43255aF66EcF0523c92B4a
Description	HBAR / USD
Decimals	8
Sample answer	10150000 → \$0.1015 per HBAR
Sample age at validation	~7 minutes (fresh)
RPC	https://testnet.hashio.io/api
Chain ID	296
Recommended maxAge (testnet demo)	3600 (1 hour)

Constructor accepts feed + maxAge so the template stays flexible. In finalize(), a stale or non-positive answer must REVERT — never silently mark goal unmet.

5. Unit Handling (critical — write tests)

Hashio exposes native HBAR with **18 decimals** (wei-style). Chainlink HBAR/USD uses **8 decimals**. Store goalUsd with **8 decimals**.

```

// totalRaised: 18-decimal HBAR (msg.value)
// answer: int256 with feed.decimals() == 8
// goalUsd: 8-decimal USD

uint8 dec = feed.decimals();
require(dec == 8, "unexpected decimals");
require(answer > 0, "bad price");

```

```
// usd (8 decimals) = hbar (18) * price (8) / 1e18
uint256 usdRaised = (totalRaised * uint256(answer)) / 1e18;
bool met = usdRaised >= goalUsd;

// Example: 100 HBAR (100e18) * 0.1015 (10150000) / 1e18
//           = 10150000000 → $10.15 (8 decimals)
```

Minimum test: mock feed; contribute known HBAR; finalize with price on each side of the goal boundary; assert goalMet.

6. Security Requirements

- **Reentrancy:** On withdraw and refund, update state (zero contribution / set withdrawn) BEFORE sending HBAR.
- **Checks-effects-interactions** on every value transfer.
- **finalize once:** require(!finalized); set finalized = true on the only success path.
- **No contribute after deadline or after finalize.**
- **Organizer-only withdraw.**
- **Oracle:** answer > 0; block.timestamp - updatedAt <= maxAge; answeredInRound >= roundId.
- Prefer strict CEI; do not pull in heavy frameworks unless already in the stack.

7. Repository Architecture

Scaffold-HBAR-compatible monorepo. Preferred stack: **Hardhat + Next.js** (official). Vite + React is acceptable if template.json / README are consistent. Node >= 20.18.3.

```
usd-goal-crowdfund/
├── package.json           # workspaces: ["packages/*"]
├── template.json          # REQUIRED scaffold-hbar manifest
├── LICENSE                # MIT
├── README.md
├── AGENTS.md
├── .gitignore
├── packages/
│   ├── hardhat/
│   │   ├── package.json
│   │   ├── hardhat.config.ts    # hedera_testnet, chainId 296, Hashio
│   │   ├── .env.example         # HARDHAT_PRIVATE_KEY, HASHIO_RPC_URL
│   │   ├── contracts/
│   │   │   ├── UsdGoalCrowdfund.sol
│   │   │   └── interfaces/AggregatorV3Interface.sol
│   │   ├── scripts/deploy.ts
│   │   ├── test/UsdGoalCrowdfund.ts
│   │   ├── deployments/ or export for frontend
│   │   └── nextjs/ (or vite frontend)
│   │       ├── package.json
│   │       ├── .env.example
│   │       ├── app or src/
│   │       │   ├── config/chains.ts
│   │       │   ├── components/
│   │       │   │   ├── CampaignCard.tsx
│   │       │   │   ├── ContributeForm.tsx
│   │       │   │   └── ActionButtons.tsx
│   │       │   ├── hooks/useCampaign.ts
│   │       │   └── page / App.tsx
```

7.1 template.json (required fields)

```
{
  "name": "usd-goal-crowdfund",
  "description": "USD-goal crowdfunding on Hedera with Chainlink HBAR/USD and automatic refunds",
  "version": "1.0.0",
  "tags": ["crowdfunding", "oracle", "chainlink", "hbar", "refunds"],
  "repository": "https://github.com/<org>/<repo>"
}
```

8. Deploy Script

- Network: hedera_testnet. Env: HARDHAT_PRIVATE_KEY (ECDSA), optional HASHIO_RPC_URL.
- Constructor: goalUsd (e.g. 10n * 10n**8n for \$10), deadline (now+3600), feed address, maxAge 3600.
- Use high gasLimit (2.5e6–4e6) and provider gasPrice — Hedera rejects low gas price.
- Print address + HashScan contract URL; write address for frontend consumption.
- Optional: second deploy with very short deadline for refund-path demo.

```
const goalUsd = 10n * 10n ** 8n;
const deadline = BigInt(Math.floor(Date.now() / 1000) + 3600);
const feed = "0x59bC155EB6c6C415fE43255aF66EcF0523c92B4a";
const maxAge = 3600n;

const Factory = await ethers.getContractFactory("UsdGoalCrowdfund");
```

```
const c = await Factory.deploy(goalUsd, deadline, feed, maxAge, { gasLimit: 2_500_000 });
await c.waitForDeployment();
const addr = await c.getAddress();
console.log("Crowdfund:", addr);
console.log("HashScan:", `https://hashscan.io/testnet/contract/${addr}`);
```

9. Frontend (single page)

9.1 Wallet

- RainbowKit + wagmi + viem; chainId 296; RPC https://testnet.hashio.io/api.
- Show ConnectButton.

9.2 Campaign card (read-only data)

- goalUsd, totalRaised (HBAR), optional preview USD via view oracle read (not used for goalMet).
- Progress bar based on preview or HBAR estimate.
- timeLeft; after deadline show Awaiting finalize / Finalized goal met|not met.
- Flags: finalized, goalMet, withdrawn; organizer address.

9.3 Actions

Button	Behavior
Contribute	HBAR amount input → payable contribute(). Disabled if !isOpen.
Finalize	Shown when past deadline and !finalized. Anyone.
Withdraw	Shown when finalized && goalMet && !withdrawn and connected wallet is organizer.
Refund	Shown when finalized && !goalMet && user contribution > 0.

No multi-page app, no token association UI, no swap UI, no second token balances.

10. Tests (minimum set)

- Mock AggregatorV3Interface. Contribute → warp past deadline → finalize with price meeting goal → withdraw OK.
- Same with failing price → refund OK; second refund reverts; organizer withdraw reverts.
- contribute after deadline reverts; finalize before deadline reverts; double finalize reverts.
- After refund, contributions[user] == 0 and user received amount.
- Boundary: usdRaised == goalUsd ⇒ goalMet true.
- Optional manual script: finalize against live testnet feed (not CI-blocking).

11. Demo Path (document in README)

1. Fund deployer (Hedera Portal faucet or existing account). New addresses need a first inbound HBAR or deploy fails with Sender account not found.
2. Deploy with small goal (\$1–\$10) and short deadline (15–60 min).
3. Use organizer + two contributor accounts; contribute various HBAR amounts from contributors.
4. After deadline, Finalize via UI or script. Observe goalMet.
5. Withdraw (organizer) or Refund (each contributor). Capture HashScan links for deploy, contribute, finalize, withdraw/refund.

Builder needs: funded deployer; ideally three funded addresses for the live demo. Contributors only need testnet HBAR. No oracle API key.

12. README Sections (required)

- One-command scaffold line.
- What this is / real-world use / how it differs from price-triggered escrow.
- Prerequisites: Node version, testnet account, faucet.
- Env var table (HARDHAT_PRIVATE_KEY, RPC, frontend public address).
- Deploy steps + constructor parameter meanings.
- UI walkthrough: contribute → wait → finalize → withdraw or refund.
- Contract + oracle addresses table; verified HashScan transaction link.
- Unit notes (18-decimal HBAR, 8-decimal USD) and volatility disclaimer on finalize price.
- Architecture diagram or short package map.

13. AGENTS.md Guidance

Tell AI agents: one campaign per deploy; never add token association; always use finalize for the only oracle read that affects goalMet; use CEI on refund/withdraw; feed address and chainId 296 are fixed for testnet demos; do not introduce SaucerSwap or WHBAR.

14. Stretch (only if core is done)

- HCS receipt: after finalize, post a compact JSON {goalMet, totalRaised, price, updatedAt} to a topic.
- Supporter badge: HTS NFT minted to contributors after successful goalMet withdraw path.
- Neither is required for eligibility or the core demo.

15. Acceptance Checklist

#	Criterion	Pass?
1	npm create scaffold-hbar@latest --template org/repo scaffolds cleanly	
2	npm install and build succeed; no secrets in git	
3	template.json, README.md, AGENTS.md, MIT LICENSE present	
4	Contract deploys on Hedera testnet; HashScan link in README	
5	contribute works with plain HBAR only	
6	finalize reads Chainlink feed and sets goalMet	
7	withdraw OR refund path works end-to-end on testnet	
8	Unit tests cover goal boundary + refund zeroing	
9	UI shows progress, time left, and the four actions with correct enablement	
10	No DEX / second-token / association steps in docs or code paths	

16. Implementation Order for Coding Agent

1. Scaffold monorepo (package.json workspaces, hardhat.config.ts, template.json, LICENSE, gitignore).
2. Add AggregatorV3Interface + UsdGoalCrowdfund.sol with exact semantics above; compile.
3. Write Hardhat unit tests with mock feed; get green.
4. deploy.ts to testnet; record HashScan; export address.
5. Frontend chain config + useCampaign reads + CampaignCard.
6. Contribute / Finalize / Withdraw / Refund buttons wired to wagmi write.
7. README + AGENTS.md; run eligibility checklist; tag v1.0.0.
8. Only then consider HCS stretch.

End of PRD. Implement MVP only. Oracle address and chain parameters above are authoritative for testnet.